

Reviewer Pack — Minimal Rank of Quiver Representations over Finite Fields an...

Entry 1bfa454d60f2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 20:37:04 UTC

Minimal Rank of Quiver Representations over Finite Fields and Communication Complexity Rank

Entry ID: 1bfa454d60f2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 20:37:04 UTC

1. Conjecture

Field A (mathematical branch): Quiver Representation Theory (finite fields) **Field B** (complexity object): Communication Complexity

Statement:

For any quiver representation ρ over a finite field F with q elements, the minimal rank of ρ is linearly correlated with the communication complexity rank $r(\varphi)$, such that $\min_rank(\rho) = \Theta(r(\varphi))$.

Rationale (proposer's reasoning):

Quiver representation theory offers a rich algebraic structure to study invariant properties of graphs, which could potentially provide insights into the computational complexity of certain graph problems. The minimal rank of a quiver representation captures information about the underlying algebraic structure, and it may be related to the communication complexity of graph problems through their structural similarities.

Taxonomy category: Quiver_Representation (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bbe28ac67f673a7a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a quiver representation ρ over finite field F , if the correlation coefficient between $\min_rank(\rho)$ and $r(\varphi)$ is ≥ 0.8 and the mean of $|\min_rank(\rho) - r(\varphi)|$ across all seeds is ≤ 3 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quiver representation theory" AND minimal rank over finite fields"
- "communication complexity" AND "rank of quiver representations" - "minimal rank quiver representations finite field" AND communication complexity

Top relevant hits considered: - [http://arxiv.org/abs/0906.3415v1] Quivers, Quasi-Quantum Groups and Finite Tensor Categories - [http://arxiv.org/abs/2111.02551v4] Bigraded Betti numbers and Generalized Persistence Diagrams - [http://arxiv.org/abs/2201.12319v1] Arithmetic Representation Growth of Virtually Free Groups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    augmented_matrix = [row[:] + [0] for row in matrix]
    for i in range(n):
        if matrix[i][i] == 0:
            for j in range(i+1, n):
                if matrix[j][i] != 0:
                    matrix[i], matrix[j] = matrix[j], matrix[i]
                    augmented_matrix[i], augmented_matrix[j] = augmented_matrix[j], augmented_matrix[i]
                    break
        if matrix[i][i] == 0:
            continue
        factor = Fraction(1, matrix[i][i])
        for j in range(i, n + 1):
            augmented_matrix[i][j] *= factor
        for j in range(n):
            if j != i and matrix[j][i] != 0:
                factor = -matrix[j][i]
                for k in range(i, n + 1):
                    augmented_matrix[j][k] += factor * augmented_matrix[i][k]
    return [row[n:] for row in augmented_matrix]

def rank(matrix):
    augmented_matrix = gaussian_elimination(matrix)
    rank = 0
```

```

for row in augmented_matrix:
    if any(row):
        rank += 1
return rank

def generate_quiver(n):
    quiver = {}
    for i in range(n):
        for j in range(n):
            if i != j and random.random() < 0.5:
                quiver[(i, j)] = random.randint(1, 10)
    return quiver

def generate_communication_complexity_instance(n):
    instance = [random.randint(1, 10) for _ in range(n)]
    return instance

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    min_rank_sum = 0
    communication_complexity_rank_sum = 0
    instances_tested = 0
    n_max = 0

    for n in n_values:
        quiver = generate_quiver(n)
        representation = [[quiver.get((i, j), 0) for j in range(n)] for i in range(n)]
        min_rank = rank(representation)
        min_rank_sum += min_rank
        instances_tested += n

        communication_complexity_instance = generate_communication_complexity_instance(n)
        communication_complexity_rank = sum(communication_complexity_instance)
        communication_complexity_rank_sum += communication_complexity_rank
        instances_tested += n

        n_max = max(n_max, n)

    mean_min_rank = min_rank_sum / instances_tested
    mean_communication_complexity_rank = communication_complexity_rank_sum / instances_tested
    correlation_coefficient = (instances_tested * sum(min_rank * communication_complexity_rank for _ in range(instances_tested))) / (instances_tested ** 2)

    conjecture_holds = correlation_coefficient >= 0.8 and max(abs(min_rank - communication_complexity_rank) for _ in range(instances_tested)) < 10
    counterexample = "" if conjecture_holds else "correlation_coefficient < 0.8 or mean(|min_rank - communication_complexity_rank|) > 10"

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":

```

```

import sys
seeds = [int(seed) for seed in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
results = []

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {{\"seed\": {seed}, ...{result}...}}")
    results.append(result)

mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c9c0b2a5.py", line 104, in <module>

```

    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c9c0b2a5.py", line 84, in run_trial

```

    correlation_coefficient = (instances_tested * sum(min_rank * communication_complexity_rank for min_rank, communication_complexity_rank in zip(min_rank_sum, communication_complexity_rank_sum)) / math.sqrt(instances_tested * sum(min_rank ** 2 for min_rank in min_rank_sum) * math.sqrt(instances_tested * sum(communication_complexity_rank ** 2 for communication_complexity_rank in communication_complexity_rank_sum) ** 2)

```

NameError: name 'min_ranks' is not defined. Did you mean: 'min_rank'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to a NameError, which prevented it from producing data necessary to evaluate the conjecture. | next: Debug and correct the error in the test code to ensure it can run successfully and produce the required data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	29785
2	propose	ollama_remote	glm4:latest	0	0	24336
3	propose	ollama_remote	glm4:latest	0	0	18036
4	propose	ollama_remote	glm4:latest	0	0	61727
5	preregistration	ollama_remote	glm4:latest	0	0	25502
6	novelty	ollama_remote	glm4:latest	0	0	8539
7	novelty	ollama_remote	glm4:latest	0	0	13357
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15862
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14133
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19532
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19947
12	judge	ollama_remote	glm4:latest	0	0	8750

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 259505 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/1bfa454d60f2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1bfa454d60f2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1bfa454d60f2.tar.gz` (if generated)